Software Architecture & Design (CEN 4065C)

Course

Software Architecture & Design

Prerequisite

COP 3530 — Data Structures

Core Theme

Conquering system complexity and constant change through disciplined, structured engineering practice.



Made with GAMMA

Today's 75-Minute Roadmap

01

Prerequisites & The Linear Fallacy

Slides 5–8

02

Core Software Engineering Definitions

Slides 9–18

03

Prescriptive vs. Agile Process Lifecycles

Slides 19–30

04

Analyzing 8 Systemic Historical Failures

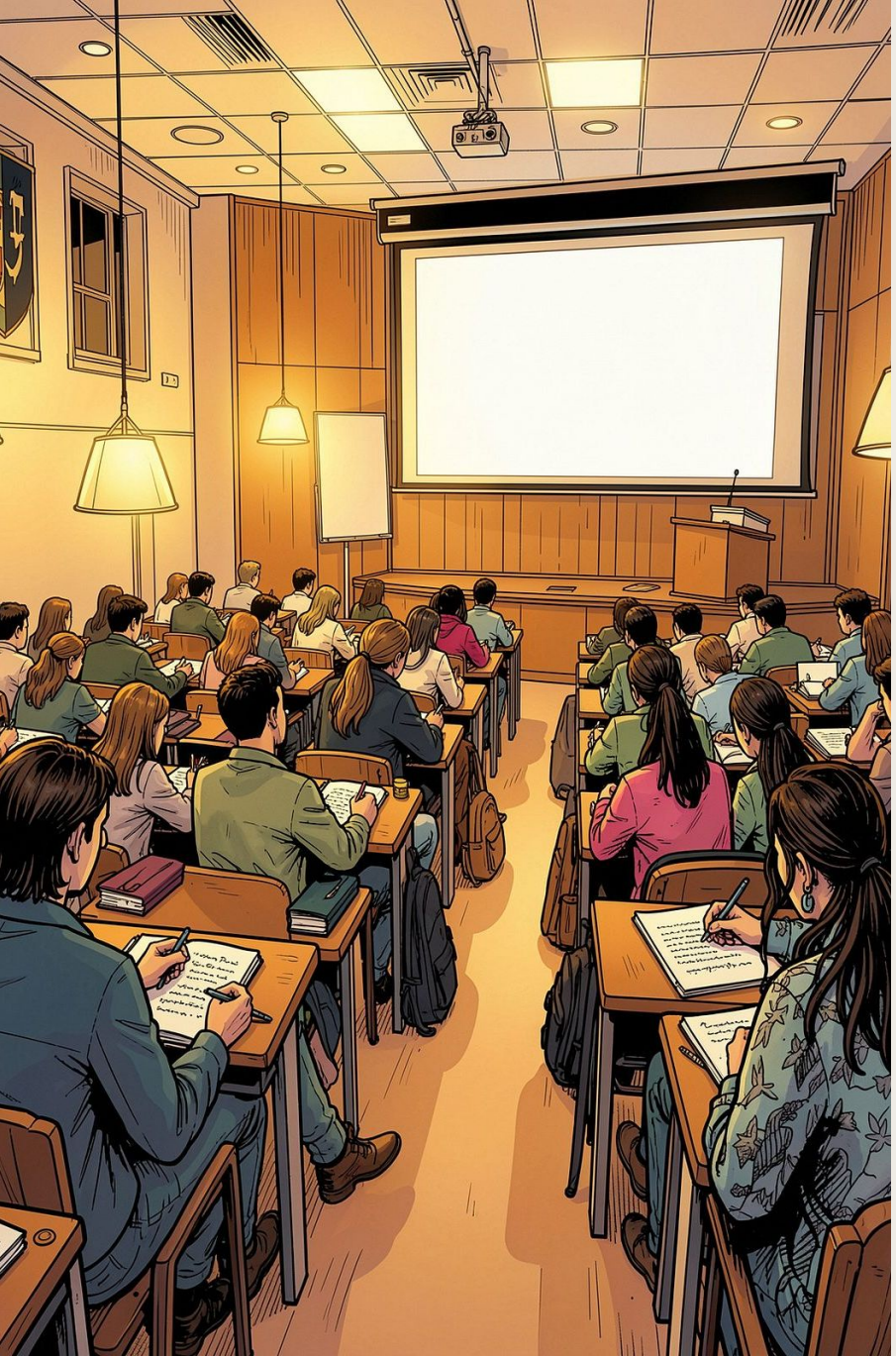
Slides 31–38

05

Trade-offs, SCM, and Formative Assessments

Slides 39–50

Made with GAMMA



Three Course Competencies Drive Today's Outcomes

1

LO1 — Process Models

Describe & contrast Waterfall, Incremental, and Evolutionary process models

2

LO2 — Agile Principles

Explain core principles behind Agile, Extreme Programming (XP), and Agile Unified Process (AUP)

3

LO3 — Human Aspects

Analyze the human aspects of development: team makeup and organization structures

Made with GAMMA



MINDSET RESET

Professional Software Engineers Abandon the Illusion of Magic

- Amateurs chase **"magic tools"** to render software trivial
- Professionals accept that **no simple panacea exists**
- Successful software requires **disciplined, structured engineering**
- Dispelling silver-bullet myths is our **first step**

Made with GAMMA

COP 3530 Establishes Our Object-Oriented Foundations



Programming Proficiency

Solid programming skills in Java or C++



Core Data Structures

Lists, Trees, Graphs, Hash Tables



OO Design Principles

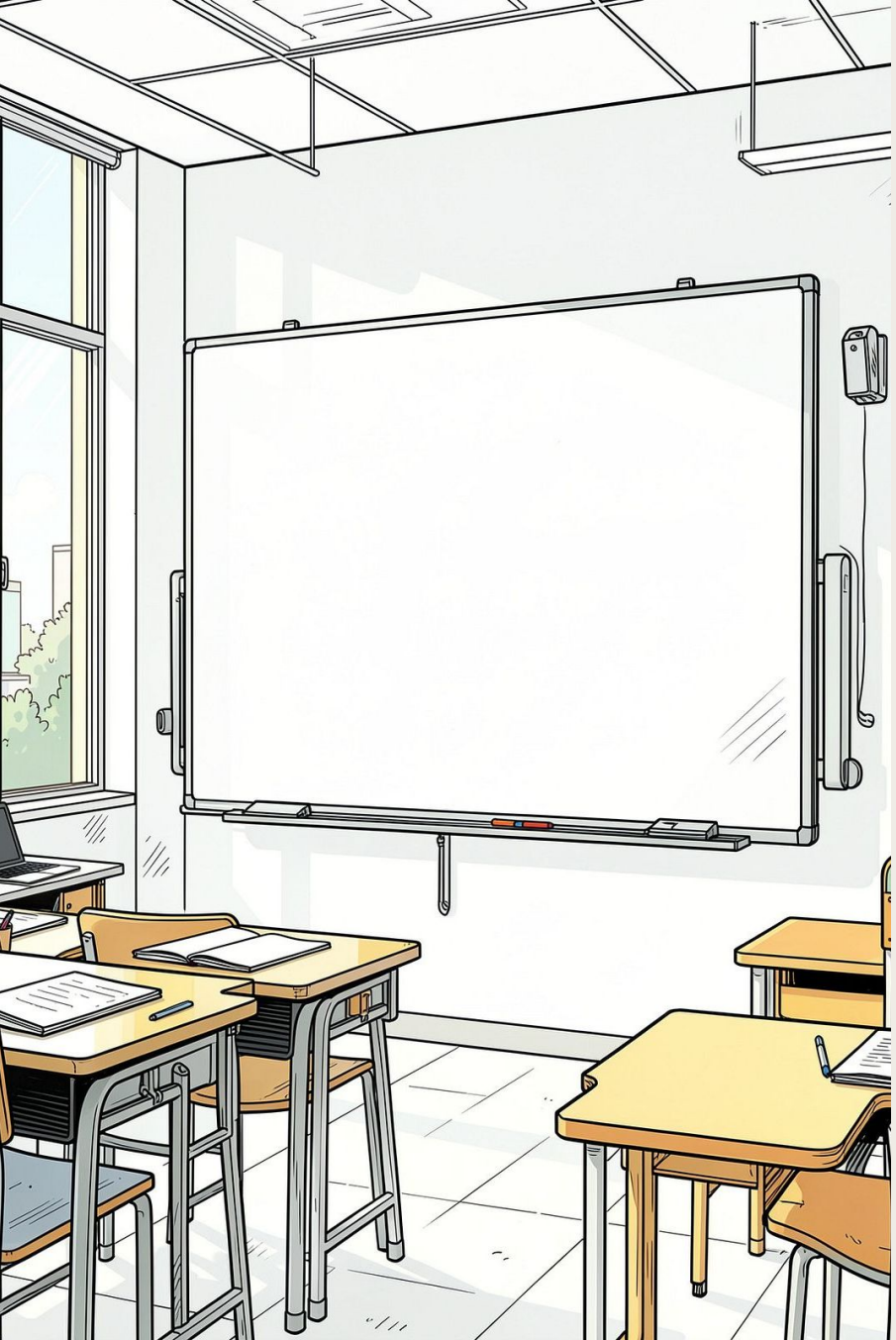
Inheritance, polymorphism, encapsulation



Engineering Basics

Debugging, unit testing, and version control

Made with GAMMA



We Start from Zero on Advanced Distributed Design

No Prior Knowledge Assumed

- Distributed systems or cloud platform experience
- Domain-driven design (DDD) background
- Formal operational or system evaluation training

What We Build Together

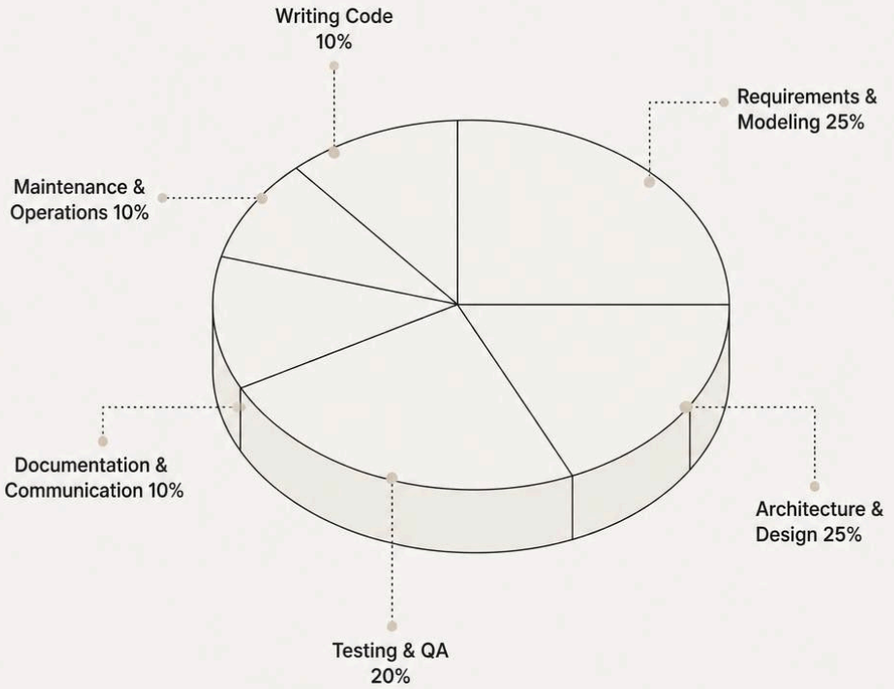
Every advanced architecture concept is introduced and built **systematically from first principles**. No student is left behind due to gaps in specialized prior experience.

Made with GAMMA

THE REALITY CHECK

Writing Code Accounts for Only 10% of Software Engineering

Code is simply **one work product among many**. Focusing purely on compilation ignores systemic architecture issues that determine whether a system succeeds or fails.



The Myth

Software engineering is just writing code quickly

The Reality

SE is a modeling, problem-solving, and knowledge-acquisition activity

Made with GAMMA



Popper's Funnel Metaphor Exposes the Linear Fallacy

The Fallacy

Information can be poured linearly into an empty project bucket

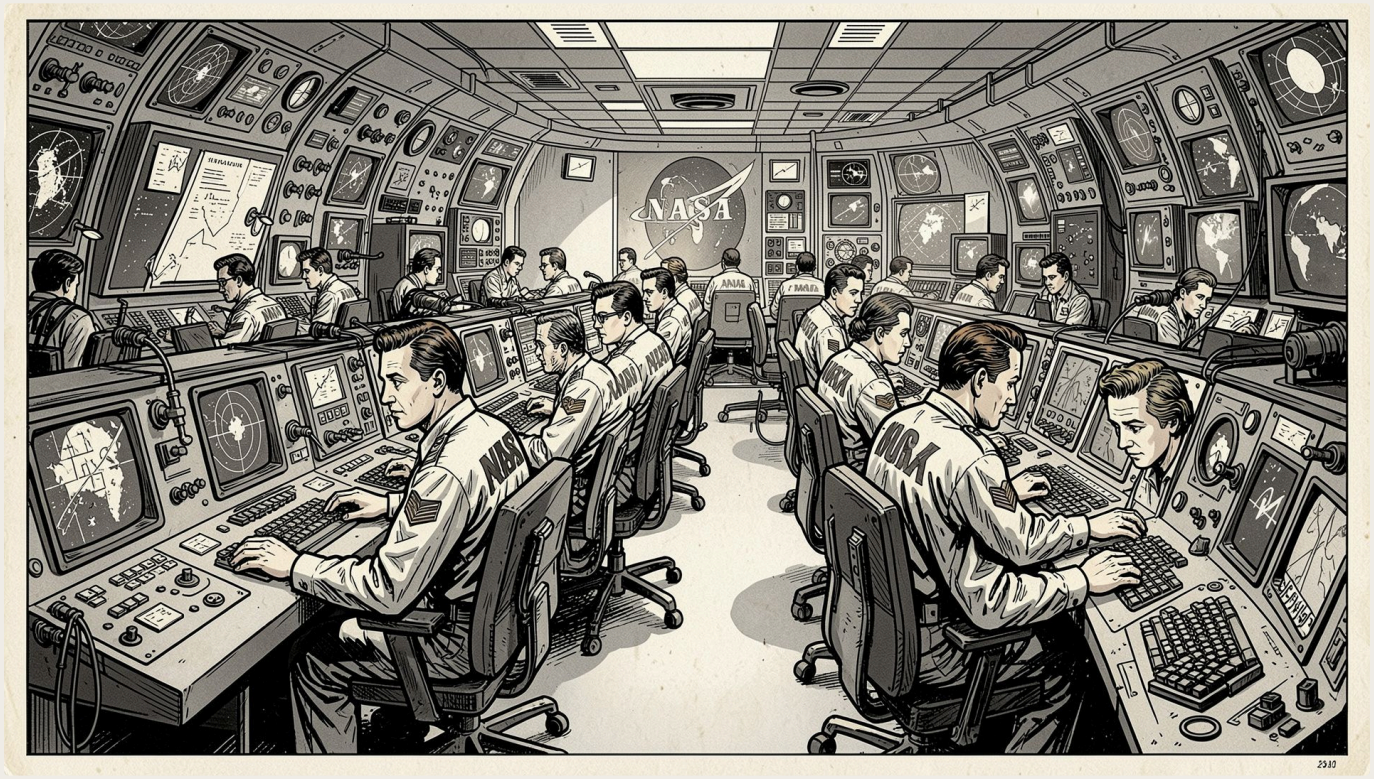
The Reality

Knowledge acquisition is **highly nonsequential** — discovery loops back and reshapes earlier decisions

The Consequence

A single late-stage discovery can **invalidate complete design models** — and linear processes have no mechanism to recover

Made with GAMMA



HISTORICAL CONTEXT

The Crisis of 1968 Demanded a Formal Engineering Shift

The Term Is Coined

1968 — coined to address the **"desolate state"** of software quality

Projects in Chaos

Traditional projects failed to establish concrete objectives or control budgets

The Tragedy

The moon was promised, but **square wheels were delivered**

The Engineering Response

Integrate off-the-shelf components under strict limits — discipline over improvisation

Made with GAMMA

Four Interlocking Perspectives Define Software Engineering

Modeling

Focus on relevant details while ignoring irrelevant ones

Problem-Solving

Search for solutions under resource constraints

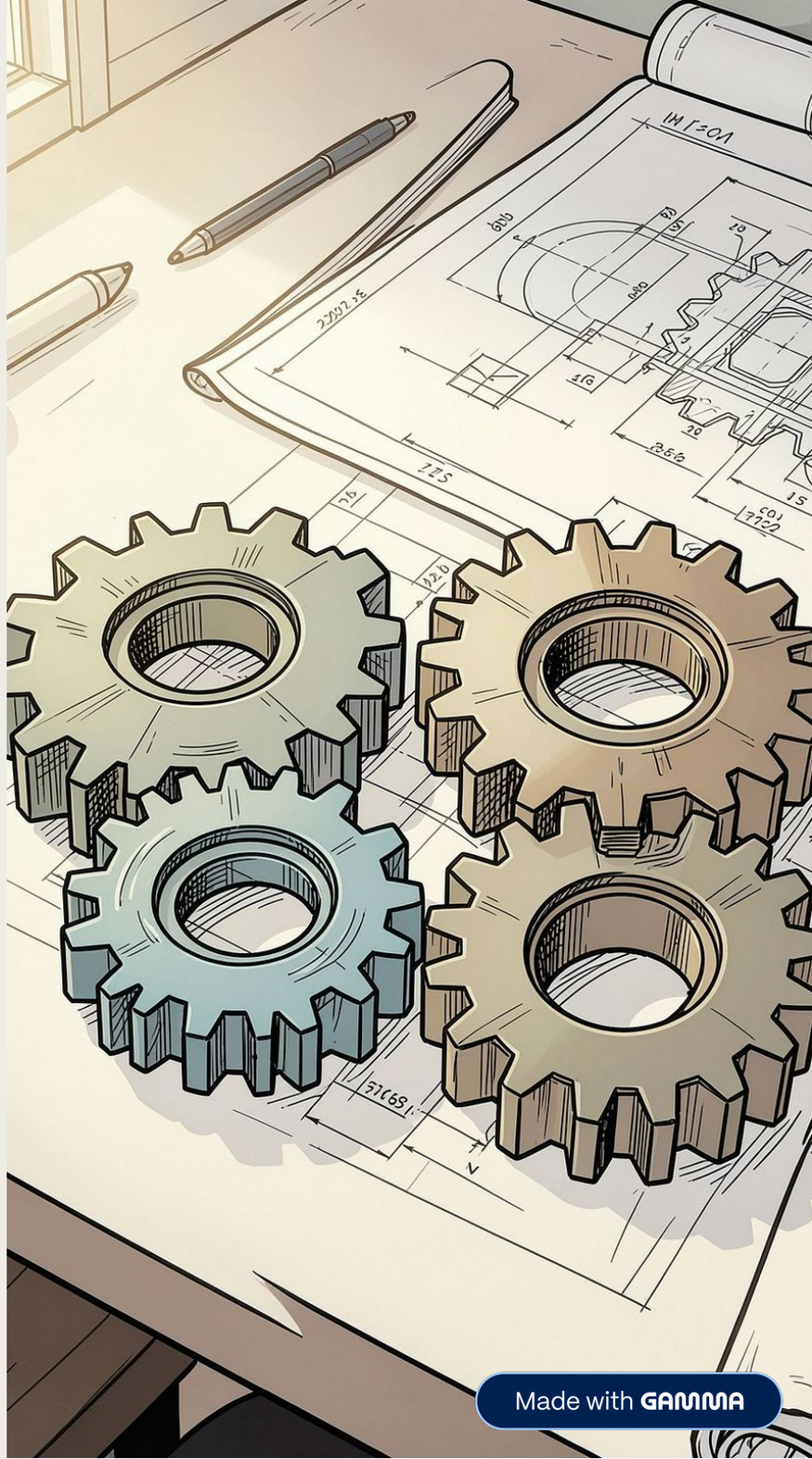
Knowledge Acquisition

Formulate and organize domain concepts nonlinearly

Rationale-Driven

Capture decision context and design justifications

Speaker notes: Software engineering is defined across these four activities. We model to ignore noise, solve problems via trial and error, acquire knowledge, and record our design rationale.



Made with GAMMA

Abstract Models Systematically Predict Physical System Behaviors



Speaker notes: A system is what we build. A model is an abstraction we construct to answer questions about that system. Schedules and budgets are models of the project system.

System

A collection of interconnected parts

Model

An abstract representation that answers questions about a system

Abstraction

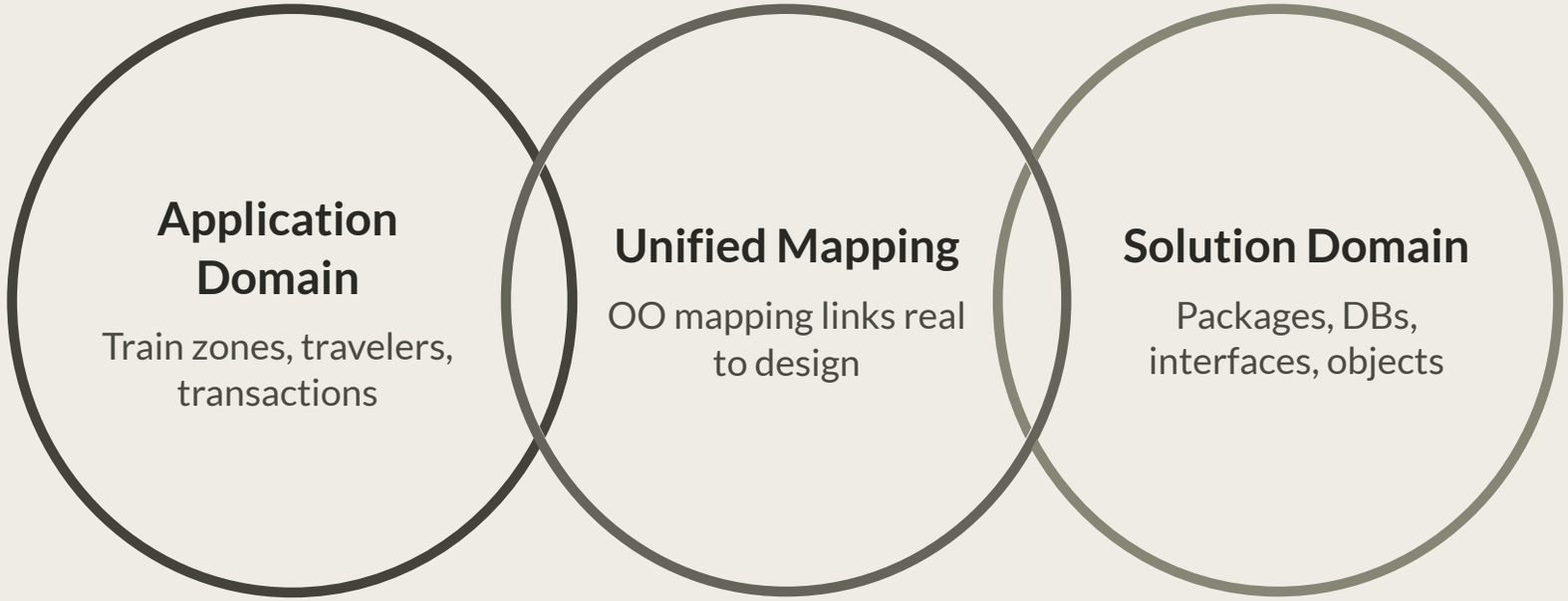
Ignoring irrelevant details to manage complexity

Examples

UML diagrams, electrical schematics, project budgets

Made with GAMMA

Object-Oriented Design Directly Maps Application to Solution Objects



- **Application Domain:** The real-world environment containing train zones and travelers
- **Solution Domain:** The designed system of packages, databases, and interfaces
- **Unified Mapping:** OO designs bridge domains by mapping concepts to objects
- *Direct Transformation:* Real-world transactions translate directly into software objects

Speaker notes: The application domain is the user's world. The solution domain is our technical architecture. Object-oriented design translates application concepts directly into code objects.

Made with GAMMA

UML Classes Structure the Project Metamodel Hierarchy

1 **Project**

Structured to deliver a system under constraints

2 **Activity**

A set of tasks toward a specific phase (e.g., Testing)

3 **Task**

An atomic, assignable unit of managed work

4 **Work Product**

Any document, model, or code produced

5 **Resources**

Participants, Time, or Equipment

Speaker notes: Our textbook organizes projects using this metamodel. A project is made of activities, which break into assignable tasks. Tasks consume resources and produce work products.

Made with GAMMA

Clear Role Assignments Prevent Project Boundary Collisions

Participant

A physical individual involved in the project

Role

A set of responsibilities associated with tasks

Client

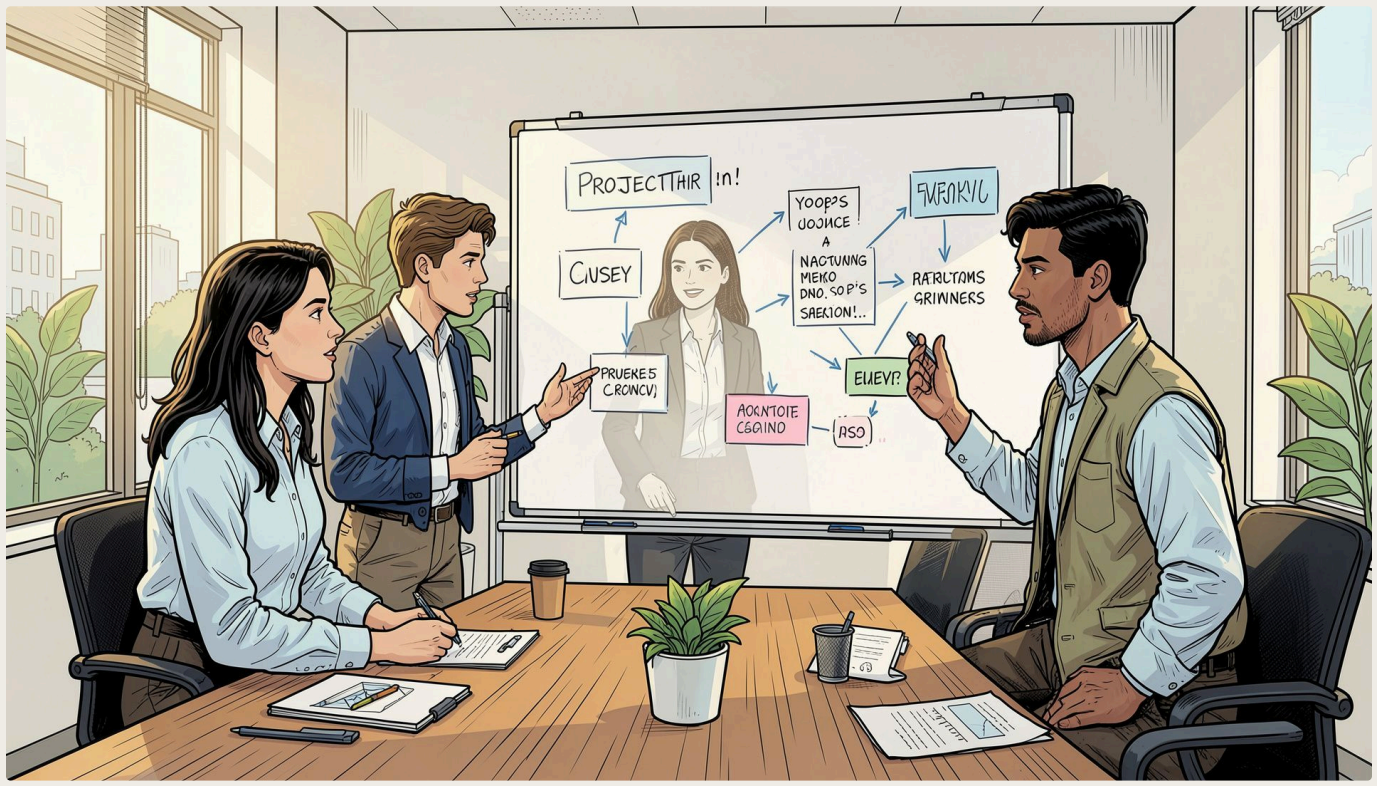
Establishes scope, budget, and high-level criteria

User

Provides domain knowledge about active user tasks

Developer

Specifies, designs, implements, and tests code

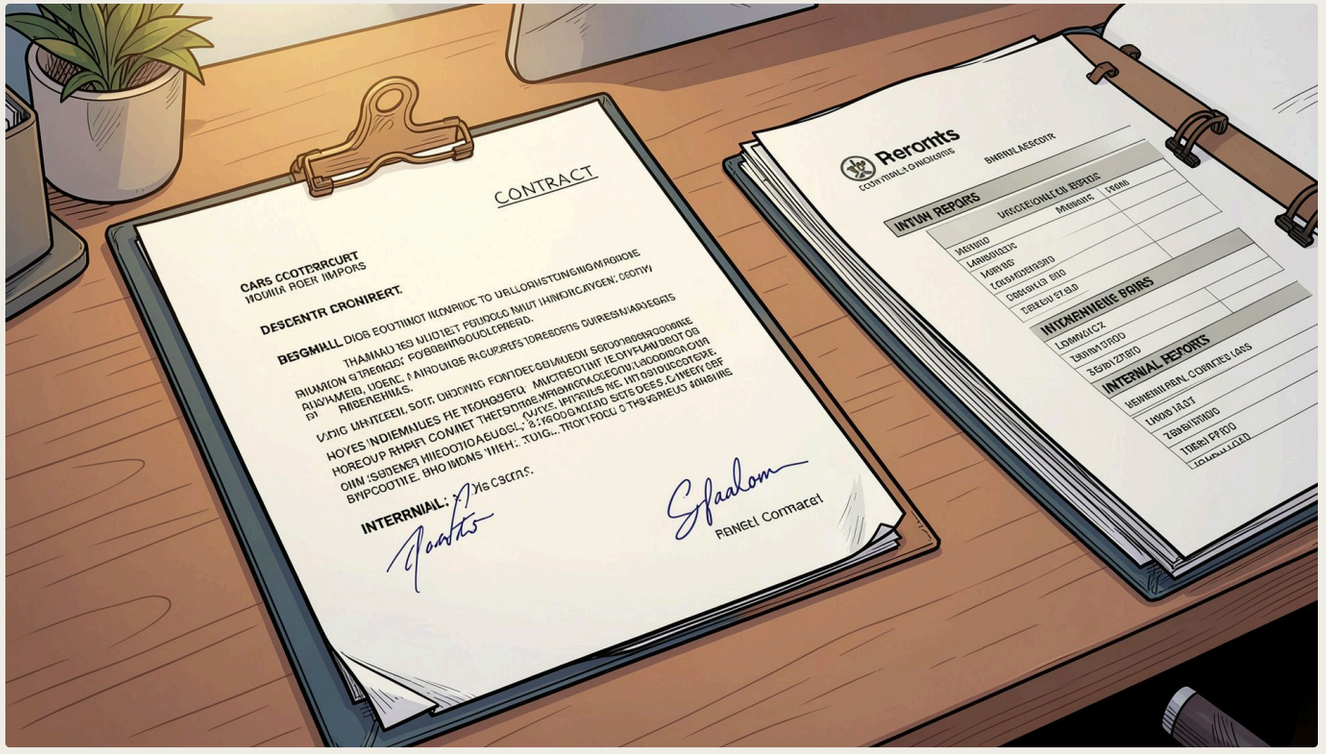


Speaker notes: We must separate participants from roles. A role is a set of responsibilities. Crucially, the Client pays for the project, while the User actually operates it.

Made with GAMMA

Contracts Differentiate Project Deliverables from Internal Products

Speaker notes: Not all work products go to the client. Deliverables are contractually specified for the client. Internal products, like status reports, are used to coordinate the team.



Deliverable

Contractually bound; delivered directly to the client

Examples: Software Specification, Operation Manual

Internal Work Product

Consumed strictly within the project team

Examples: Status Reports, Test Manuals

Made with GAMMA



Constraints Diverge from Core Functional Requirements

1	**Functional Requirements** Specify behaviors or features the system must support *Example: "Buy weekly passes"*	1	**Nonfunctional Requirements** Specify operational constraints or quality limits *Examples: "Feedback < 1s", "Write in Java"*

Speaker notes: Requirements dictate our goals. Functional requirements define what the system does. Nonfunctional requirements specify quality constraints like performance, security, and platforms.

Made with GAMMA

Methodologies Assemble Methods Using Unified Visual Notations



Notation

Graphical or textual rules for representing models

e.g., UML — the Alphabet



Method

A repeatable technique for solving a single problem

e.g., Sorting — the Recipe



Methodology

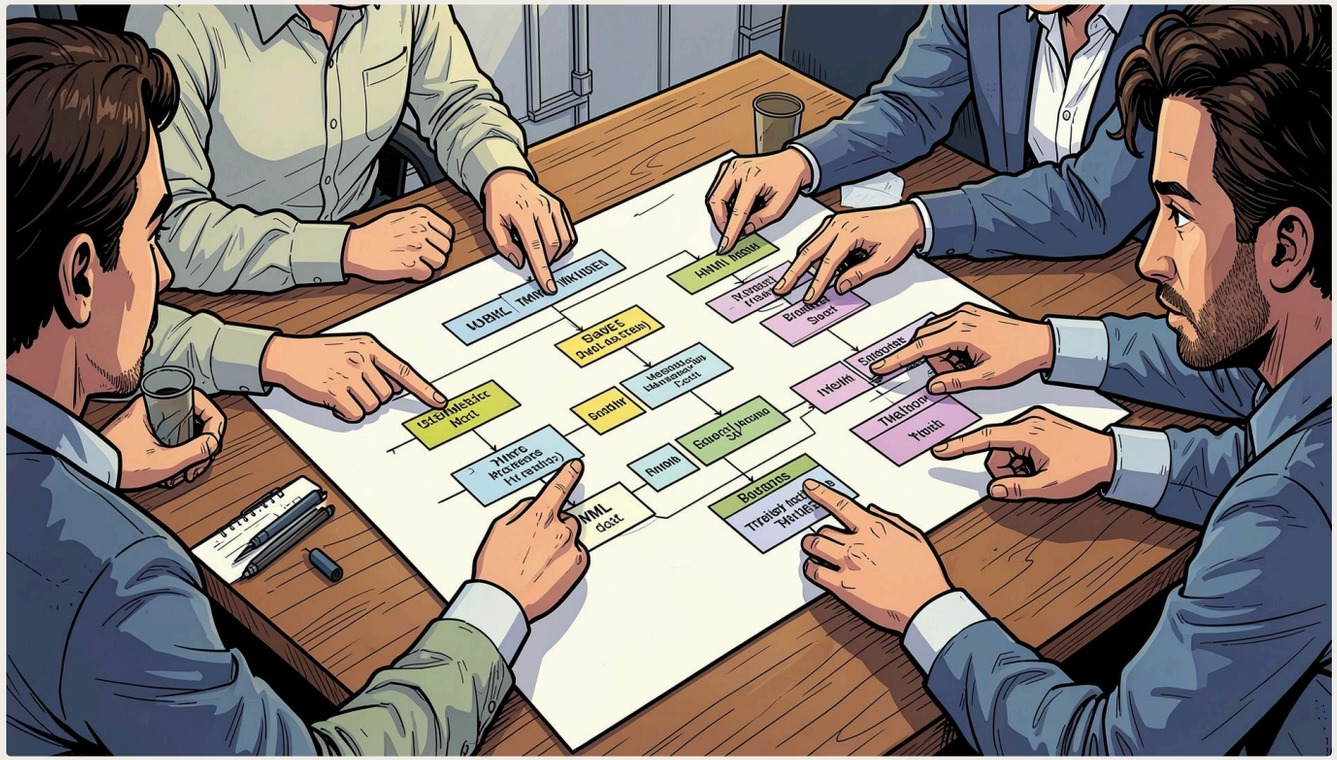
A collection of methods applied to a class of problems

e.g., Unified Process — the Cookbook

Speaker notes: Don't confuse these terms. UML is a notation. Sorting is a method. A methodology, like the Unified Process, is the complete cookbook directing our activities.

Made with GAMMA

UML Standardizes Visual Communication Across Complex Teams



- UML is our visual language to model OO software structures
- Enables communication with clients, users, and developers
- UML diagrams act as a shared team convention to prevent errors
- *Static Views:* Class diagrams. *Dynamic Views:* Sequence diagrams.

Speaker notes: UML is our primary modeling notation. It serves as a visual language that helps developers, clients, and managers communicate design concepts unambiguously.

Static Views

Class Diagrams — structure at rest

Dynamic Views

Sequence Diagrams — behavior over time

Made with GAMMA

Process Models Establish Predictable Software Life Cycles

Software Life Cycle

A general model representing the structured steps through which software is conceived, developed, and retired. Provides a shared vocabulary for teams, managers, and stakeholders.

Prescriptive Process Models

Prescriptive models impose a structured, orderly roadmap on the development effort. Their primary goal is to make outcomes predictable across three dimensions:

- **Quality** — consistent standards enforced at each phase
- **Budgets** — cost estimates tied to defined milestones
- **Milestones** — measurable checkpoints for progress tracking

Waterfall

Linear, sequential phase execution

Incremental

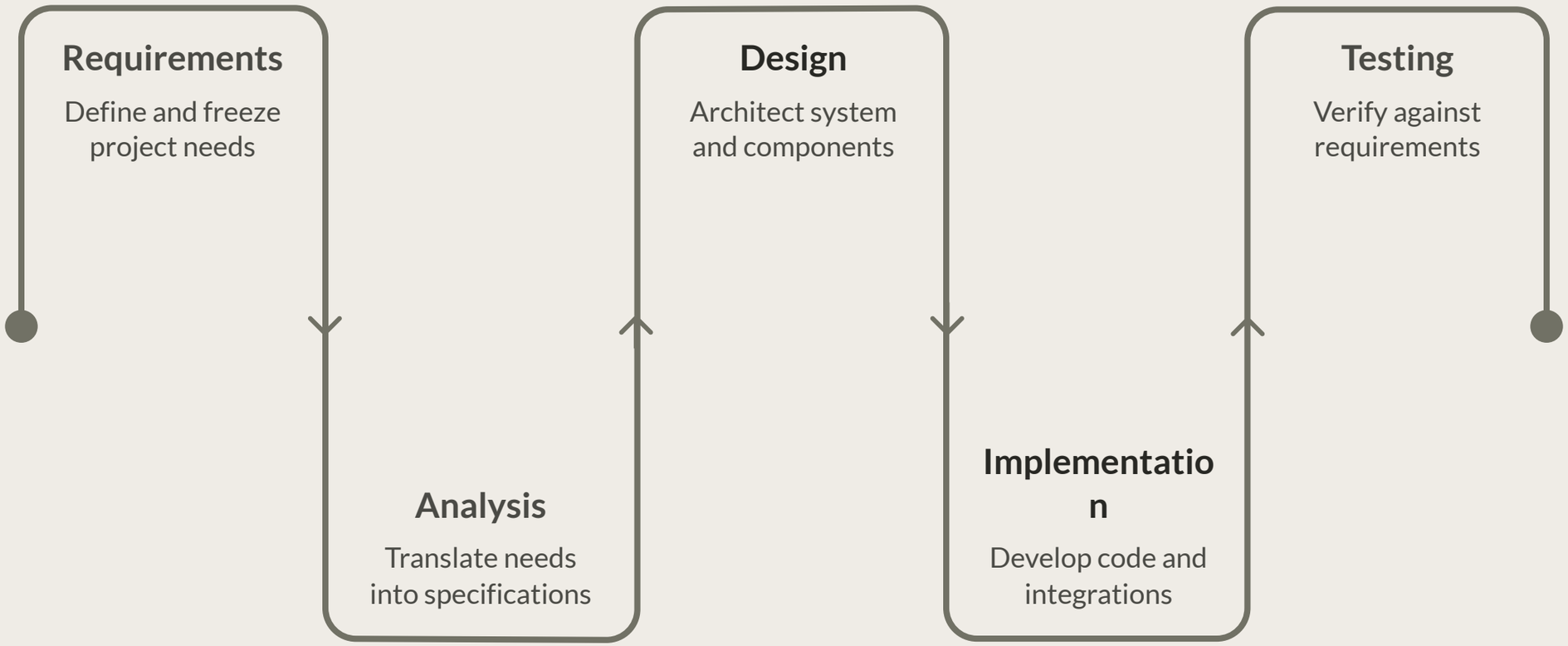
Staged delivery of functional builds

Evolutionary

Organic growth through feedback loops

Made with GAMMA

Freezing Linear Requirements Dooms the Classic Waterfall



Model Definition

Waterfall Model: All engineering steps run in a strict linear, sequential order. Each phase must be fully completed and signed off before the next phase begins. There is no formal mechanism for revisiting earlier phases once they are closed.

Core Vulnerability

- Assumes requirements can be **frozen upfront** — a condition rarely met in practice
- Highly susceptible to **late-stage integration surprises** when subsystems are assembled for the first time

⚠ The Waterfall model's linear rigidity makes it fundamentally ill-suited for projects where requirements evolve — which describes the majority of real-world software efforts.

Made with GAMMA

Incremental Processes Deliver Value Through Small Verified Builds

Model Definition

Incremental Model: Software is delivered to the client in small, self-contained builds rather than as a single monolithic release. Each build implements and tests a **functional subset of requirements**, producing a working product increment that can be evaluated immediately.

Key Structural Properties

- Each increment is independently testable and deployable
- Avoids massive, high-risk late-stage system integration
- Allows developers to validate assumptions in real environments before committing to subsequent increments

Why Incremental Wins on Risk

The single greatest source of cost overrun in software projects is late-stage integration failure — discovering that independently developed components do not work together. The incremental model eliminates this by integrating continuously, keeping risk exposure small and bounded at every stage.

Build 1

Core functional subset — requirements gathered, implemented, and tested

Build N

Each subsequent increment adds verified functionality on a stable base

Final Release

Complete system assembled from validated, integrated increments

Made with GAMMA

Evolutionary Prototyping Resolves Volatile Domain Ambiguities

Model Definition

Evolutionary Model: The system grows organically through continuous feedback between developers and users. Unlike waterfall, requirements and design models are developed *concurrently* rather than sequentially — each informing and refining the other.

Mechanism: Rapid Prototyping

The evolutionary model relies on rapid prototyping to clarify **application domain concepts** that are poorly understood at project outset. Users interact with early, incomplete prototypes to:

- Discover errors in requirements before they are codified
- Surface implicit assumptions and unstated needs
- Validate that the emerging system matches real operational intent

Requirements

Partially specified; refined through use



Prototype

Rapid build for user evaluation

Refinement

Requirements and design updated concurrently

Feedback

User interaction reveals errors and gaps

Made with GAMMA

Context-Sensitive Trade-offs Govern Prescriptive Process Choices

No single process model is universally optimal. Selection must be driven by the specific constraints, risk profile, and change tolerance of the system under development.

Model	Best Fit	Change Handling	Risk Profile
Waterfall	Stable, highly regulated requirements	Poor — changes are costly once phases close	High late-stage integration risk
Incremental	Staged feature rollouts	Moderate — new requirements enter future increments	Risk bounded per increment
Evolutionary	Exploring ambiguous domains	Excellent — change is the expected input	Excellent early risk discovery

No Universal Best: Process selection must match system constraints. Applying the wrong model to a project context is itself a significant source of project failure.

Made with GAMMA

Four Agile Values Prioritize Human Interactions and Code Delivery

Agile's Core Stance

Adaptability: Agile methods are built around welcoming changing requirements — even late in development. This is a deliberate inversion of the prescriptive model assumption that requirements can be frozen.

The Four Agile Values

The Agile Manifesto establishes four explicit value trade-offs. Each prioritizes the left-hand item *over* the right-hand item — not by eliminating the right, but by weighting the left more heavily.

1

Value 1: Individuals & Interactions

Over processes and tools

2

Value 2: Working Software

Over comprehensive documentation

3

Value 3: Customer Collaboration

Over contract negotiation

4

Value 4: Responding to Change

Over following a plan

Made with GAMMA

Extreme Programming Maximizes Adaptability Through Test-First Practices

Model Definition

Extreme Programming (XP): An agile method that takes proven software engineering practices and pushes them to extreme levels of application. XP is not a collection of new ideas — it is the disciplined, consistent application of practices already known to improve quality.

Why "Extreme"?

If code review is good, XP does it continuously — via pair programming. If testing is good, XP does it first — before production code exists. The "extreme" label reflects intensity of application, not novelty of technique.



Test-First Development

Write tests *before* writing production code. Tests define the specification; code is written to pass them.



Pair Programming

Two developers collaborate at one workstation — one writes, one reviews in real time. Continuous peer review.



Frequent Releases

Deliver small, operational software updates continuously. Feedback cycles are measured in days, not months.

Made with GAMMA

The Unified Process Bridges Enterprise Phases with Agile Speed

Model Definition

Agile Unified Process (AUP): Combines the discipline and architectural rigor of the Unified Process with the responsiveness and human focus of agile values. Designed for teams that need structure without bureaucratic overhead.

UML Usage

Uses UML notation for *communication* — not as a heavy documentation burden. Models are lightweight artifacts that facilitate shared understanding, not compliance artifacts.

Iterative Lifecycle Phases

1

Inception

Scope and feasibility

2

Elaboration

Architecture baseline

3

Construction

Iterative build cycles

4

Transition

Deployment and handoff

Each phase executes through multiple short iterations, maintaining architectural control while supporting rapid release loops.

Made with GAMMA

Broken Team Communication Overwhelms Complex Algorithms

The Foundational Premise

Software is built by **people**, for **people**. No matter how sophisticated the algorithms or architectures involved, the human communication layer is the dominant variable in project outcomes.

Communication as the Critical Path

Communication is the most critical and time-consuming project activity. It is not a soft concern peripheral to engineering — it is the engineering substrate on which all technical work depends.

Scale & Distribution Amplify Complexity

Diversity, scale, and geographic distribution each independently increase communication complexity. Together, they create combinatorial overhead that can consume the majority of project bandwidth.

Misunderstandings, Not Bad Code, Cause Defects

Misunderstandings — between developers, between developers and clients, between teams — cause most system defects. Bad coding is a secondary failure mode. Communication failure is primary.

Made with GAMMA

Four Team Roles Partition Specialized Project Responsibilities

Effective software projects require clearly delineated roles. Each role addresses a distinct dimension of the development effort — conflating them produces accountability gaps and quality failures.

Manager

Staffs the project, constructs schedules, monitors progress against milestones, and manages resource allocation. The primary interface between the technical team and organizational leadership.

Developer

Specifies, designs, implements, and validates system code. The broadest technical role — spans the full engineering lifecycle from requirements analysis through unit and integration testing.

Human Factors Specialist

Evaluates the usability of user interfaces. Ensures that the system's interaction model matches the cognitive and operational needs of its intended users — a discipline distinct from functional correctness.

Technical Writer

Compiles operational and contract documentation. Produces the artifacts that allow users to operate the system and that satisfy contractual deliverable obligations.

Made with GAMMA

Decentralized Meshes Suffer Heavy Coordinating Communication Overhead

Team topology is not merely an organizational preference — it is a direct determinant of communication load, decision speed, and creative output. The two canonical structures represent opposite trade-offs.

Hierarchical Team

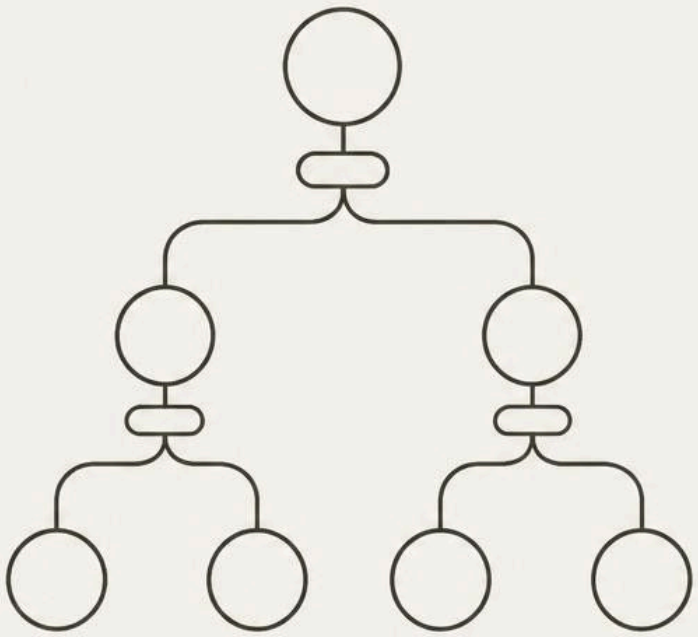
Communication is routed **vertically** through a central authority. Information flows up and down a defined chain of command rather than laterally between peers.

- *Benefit*: **High organization** — clear accountability, predictable decision paths, manageable coordination overhead
- *Risk*: **Decision bottlenecks** — the central authority becomes a throughput constraint; lateral information is filtered or delayed

Decentralized Team

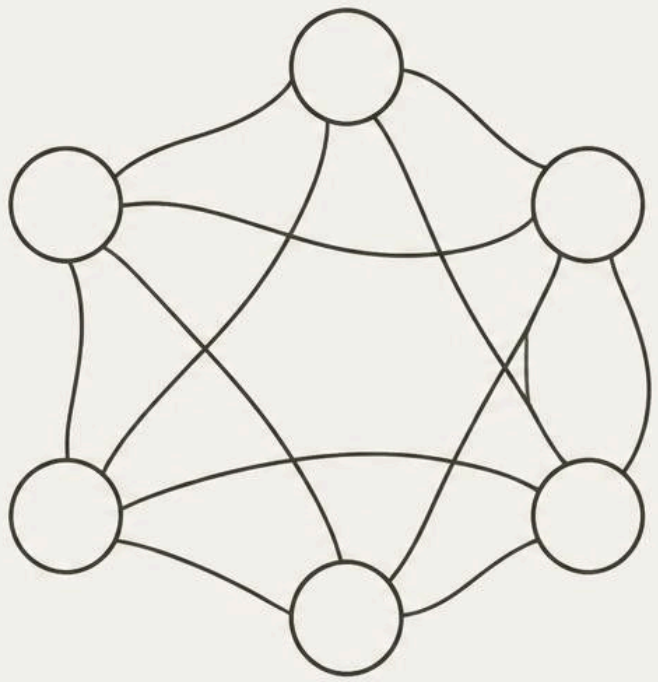
Communication flows through a **flat, peer-to-peer mesh**. Every team member communicates directly with every other — there is no designated routing authority.

- *Benefit*: **Highly creative** — unconstrained lateral communication accelerates idea generation and cross-pollination
- *Risk*: **Extremely difficult to manage** — coordination overhead scales with the square of team size; consensus is costly



Hierarchical

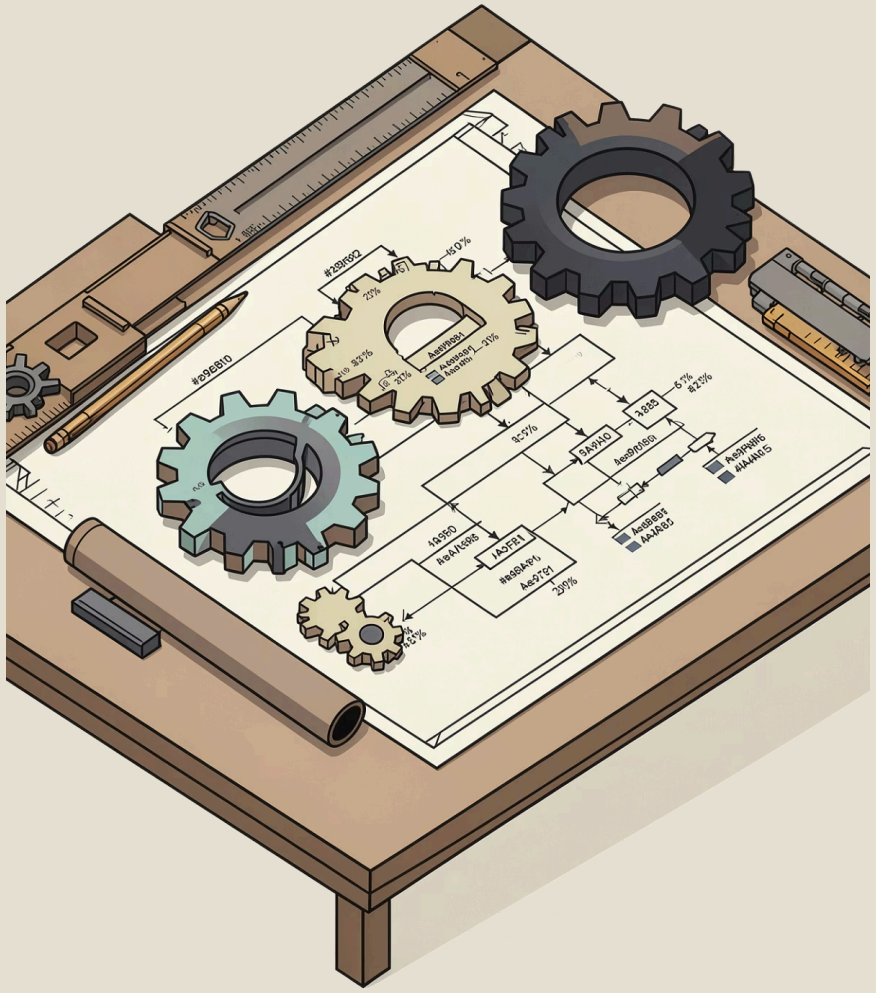
vertical chain
central authority
high organization
decision bottlenecks



Decentralized

flat peer-to-peer mesh
highly creative
extremely difficult to manage

Made with GAMMA



Process Models Systematically Streamline Iterative Lifecycles

Complex Systems

Development processes are complex systems with inputs and outputs

Software Life Cycle

A **Software Life Cycle** is our model of this process system

Analyze & Optimize

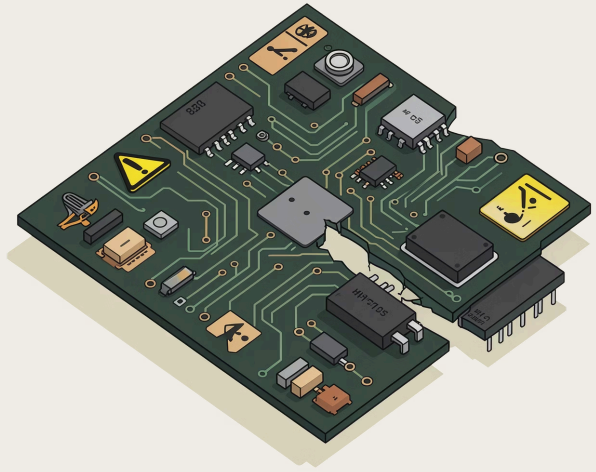
Modeling our process lets us analyze and optimize our engineering steps

Monitor & Track

Enables systematic monitoring of schedules and expended resources

Made with GAMMA

Five Mismatch Categories Classify Systemic Software Disasters



Model & Reality

System fails to represent real-world edge cases

User & System

Failure to anticipate active user misuse

Threat & Environment

Failing to model active network adversaries

Process & Management

Failures in scheduling, complexity, and budgets

Over-Engineering

Creating unnecessary structural complexity

Made with GAMMA

Two-Digit Modeling Oversights Caused Boundary Failures

The Issue: Simplifying assumptions fail to capture real-world exceptions

Year 1900 Bug (1992)

104-year-old Mary received a kindergarten invite

Cause: Two-digit year model calculated her age as 4

Leap-Year Bug (1988)

Supermarket fined $1,000 for expired meat on Feb 29

Cause: Expiration program did not model leap years



Made with GAMMA

Taped Interlock Start Buttons Expose Interface Misuse

The Issue

Developers fail to model how users will bypass safety measures

London Underground — April 10, 1990

- A driverless train left the station without its operator
- **Safety Feature**: Interlock prevented starting if doors were open
- **The Bypass**: Driver taped down start button to clear a door jam

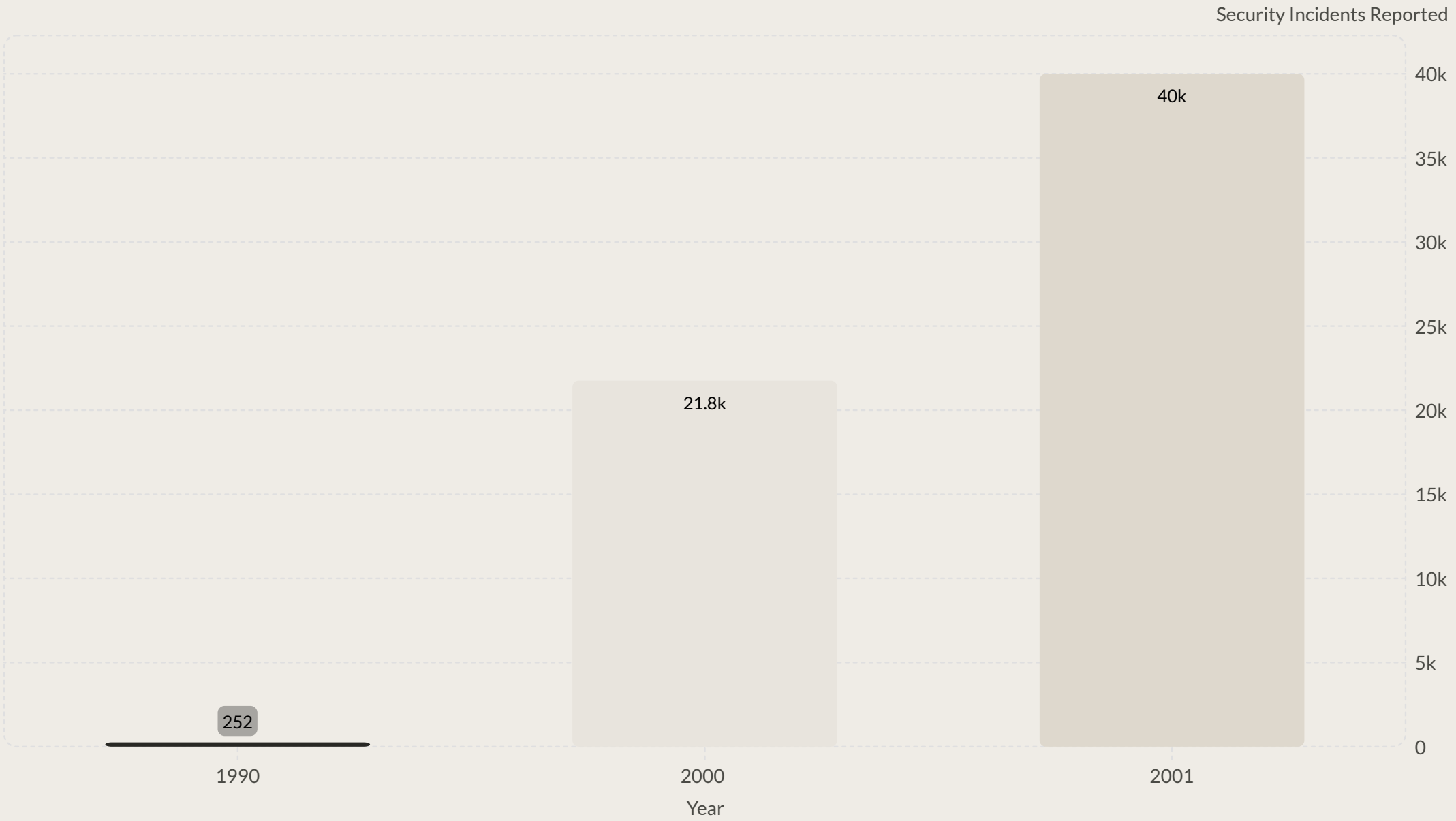


⚠ When users can circumvent safety systems, they will — and the consequences can be catastrophic.

Made with GAMMA

Global Networking Incidents Surged 15,000% Over a Decade

The Issue: Designing for isolated systems, ignoring network environments



Lesson: Security models must actively adapt to connected, hostile environments. CERT data shows a staggering rise from 252 incidents in 1990 to **over 40,000** in 2001 — a 15,000%+ escalation in a single decade.

Made with GAMMA

Wisconsin and Denver Cases Reveal Costly Management Failures

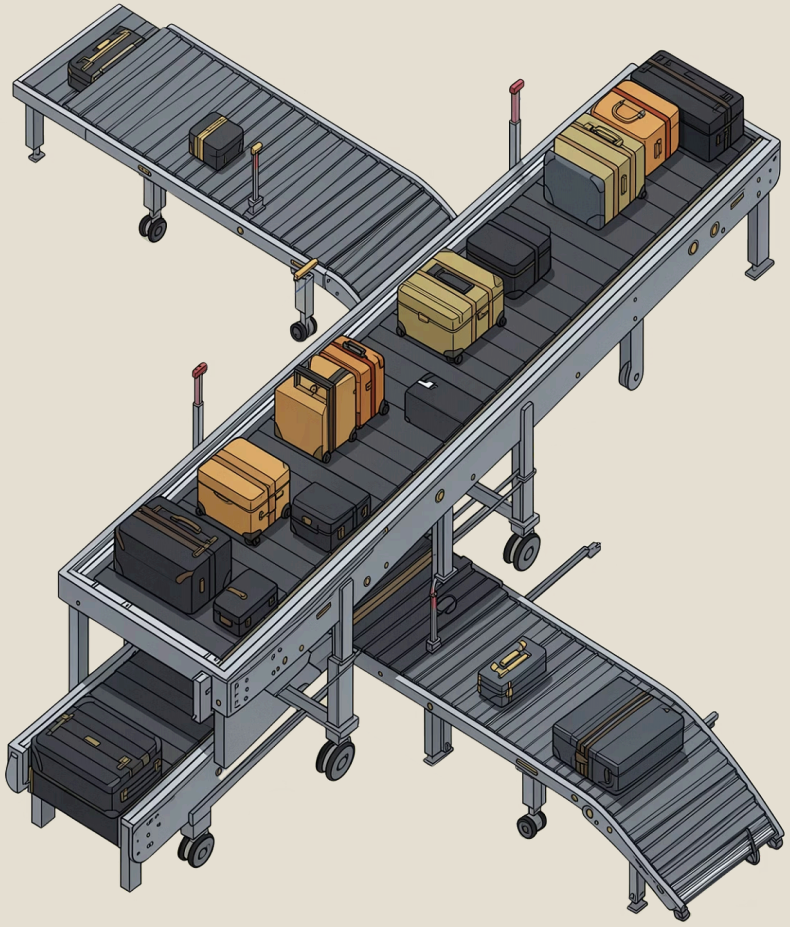
Denver Airport Luggage System (1995)

Automated cart bugs delayed opening by **16 months**, running **$3.2B over budget**

Wisconsin Health Insurance (1984)

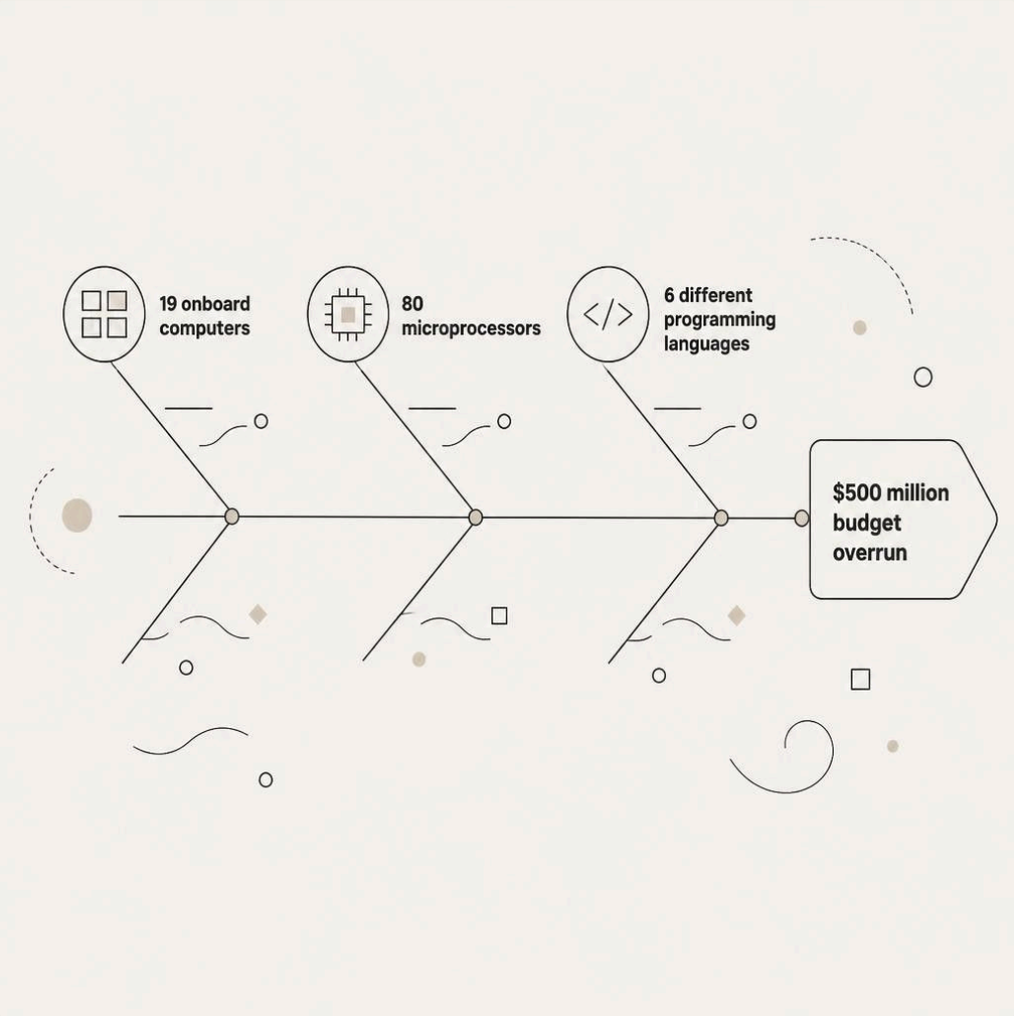
Delivered exactly on time, but issued **$60 million in overpayments**

Lesson: On-time delivery of an incorrect system is still a catastrophic failure



Made with GAMMA

Language Fragmentation Ran C-17 Avionics $500M Over Budget



The Issue

Unnecessary system complexity and technology fragmentation

C-17 Cargo Plane Avionics Software

- Ran **$500 million over budget**
- Included **19 onboard computers** and **80 microprocessors**
- Coded in **6 different programming languages**

Lesson: Enforce conceptual integrity; simplicity is an architectural driver

Made with GAMMA



TicketDistributor Projects Map Core Functional and Quality Drivers

Challenge: Build an automated ticket machine resilient to user errors and outages

Functional Requirements

Buy tickets, select single/multiple trips, calculate zones

Nonfunctional Constraints

Under-second feedback, cash outages, power failures

Analysis Goals

Build models that describe structural and dynamic system behaviors

Made with GAMMA

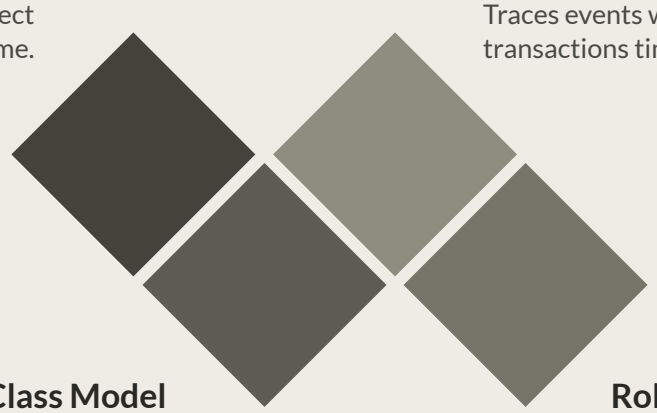
UML Sequence Diagrams Proactively Trace Abort Rollbacks

Sequence Model

Traces dynamic object messages over time.

Outage Handling

Traces events when transactions time out.



Class Model

Defines static boundaries: Ticket, Zone, Coin, Transaction.

Rollback Goal

Ensure all inserted change is returned on abort.

1

UML Class Model

Defines static boundaries — Ticket, Zone, Coin, Transaction

2

UML Sequence Model

Traces dynamic object messages over time

3

Outage Handling

Traces exact events when transactions time out

4

Rollback Goal

Ensure system returns all inserted change if a transaction is aborted

Made with GAMMA

Risk-Based Iterations Implement Critical Bottlenecks First

Since knowledge acquisition is nonlinear, we must avoid rigid linear processes

1

Risk-Based

Identify and build high-risk components first to eliminate late surprises

2

Issue-Based

Execute all lifecycle activities in parallel to resolve design debates

3

Trade-off

Parallel, nonsequential models are highly adaptive but hard to manage



Made with GAMMA

Technology Drift and Staff Turnover Force Constant Refinement

Shifting Requirements

Requirements change constantly as developers and clients refine their understanding

Training Burden

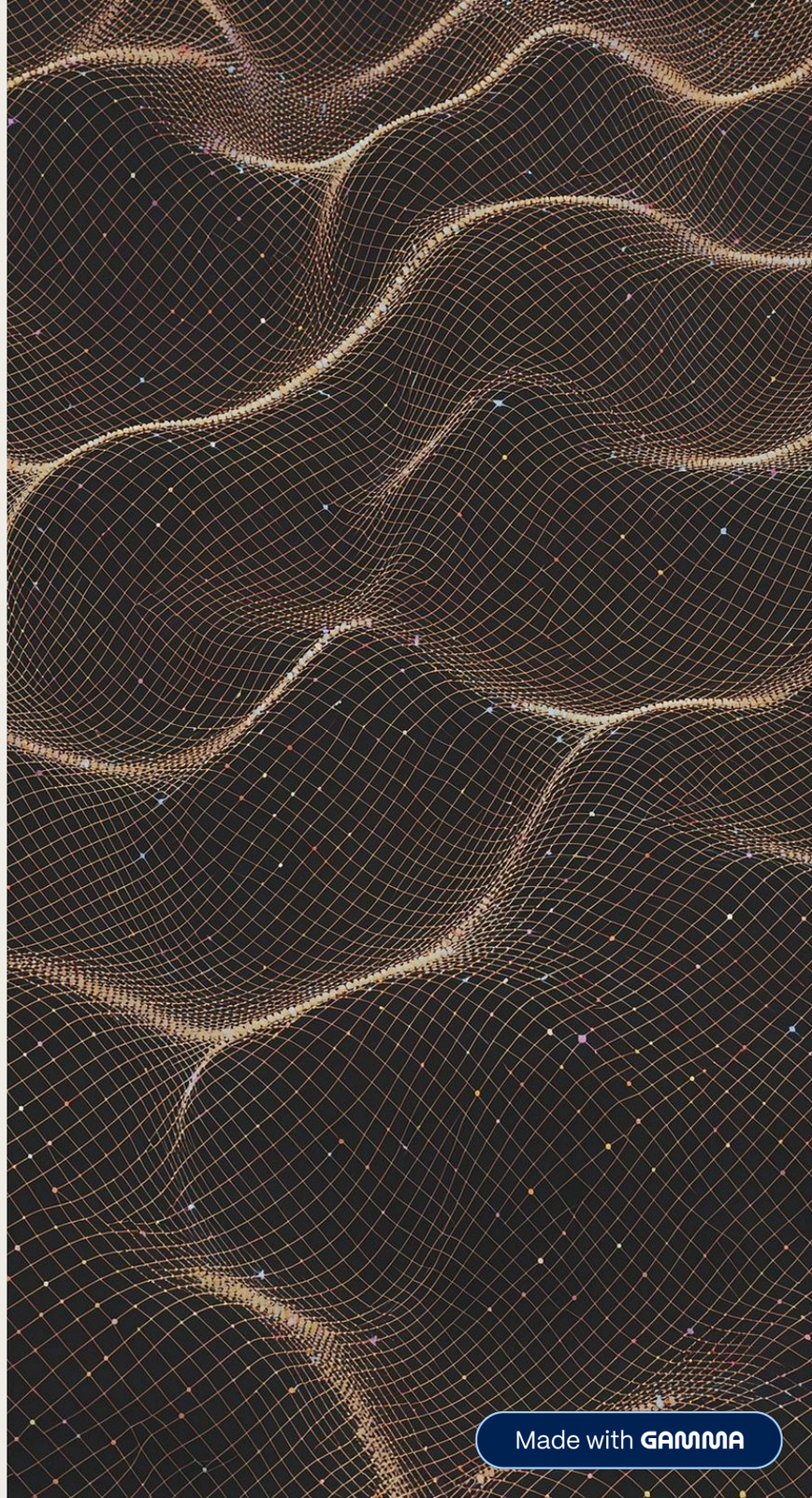
Staff turn-around on long projects creates a constant training burden

Technological Velocity

Technological changes occur faster than the duration of the project

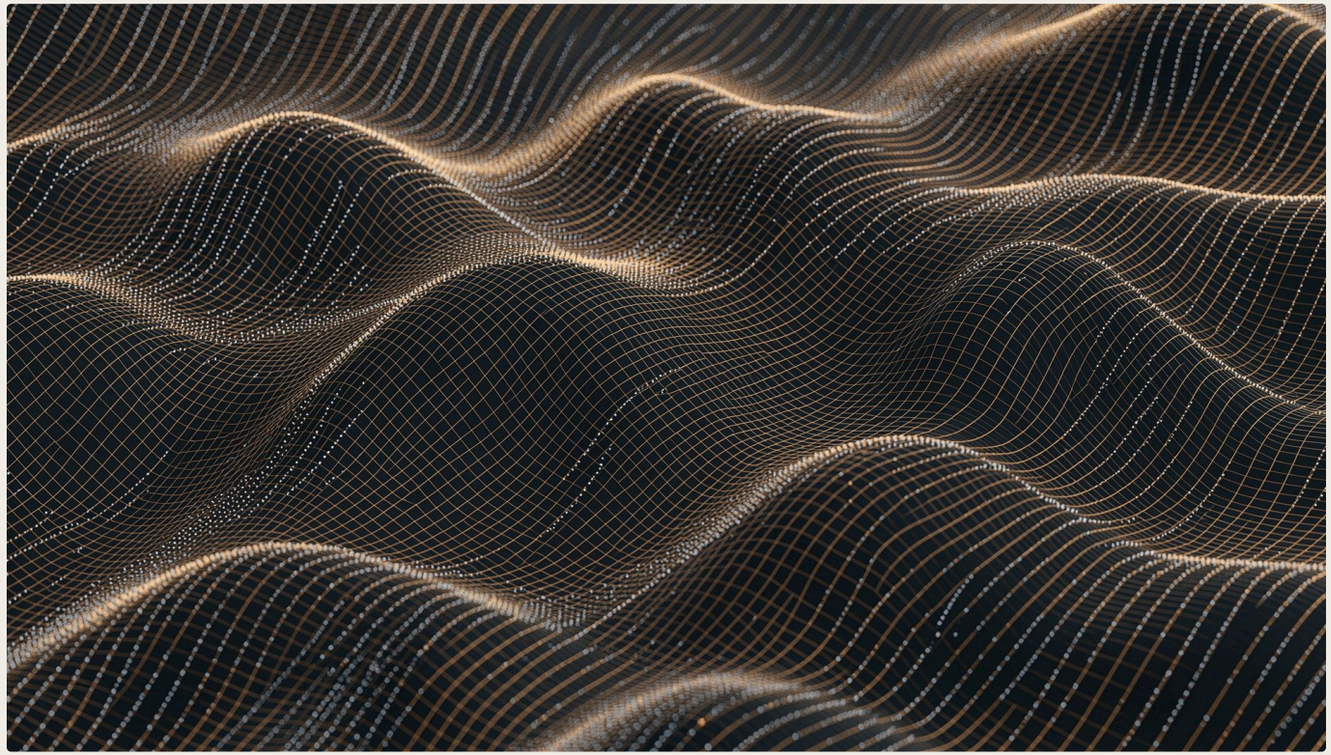
Obsolete Deployments

Freezing requirements on complex projects leads to obsolete deployments



Made with GAMMA

Shared Team Conventions Confound Coordination Overheads



Unstructured communication scales exponentially with team size. Our best defense against communication overhead is establishing **conventions**.

> Conventions do not need to be perfect; they simply must be **shared by everyone**.

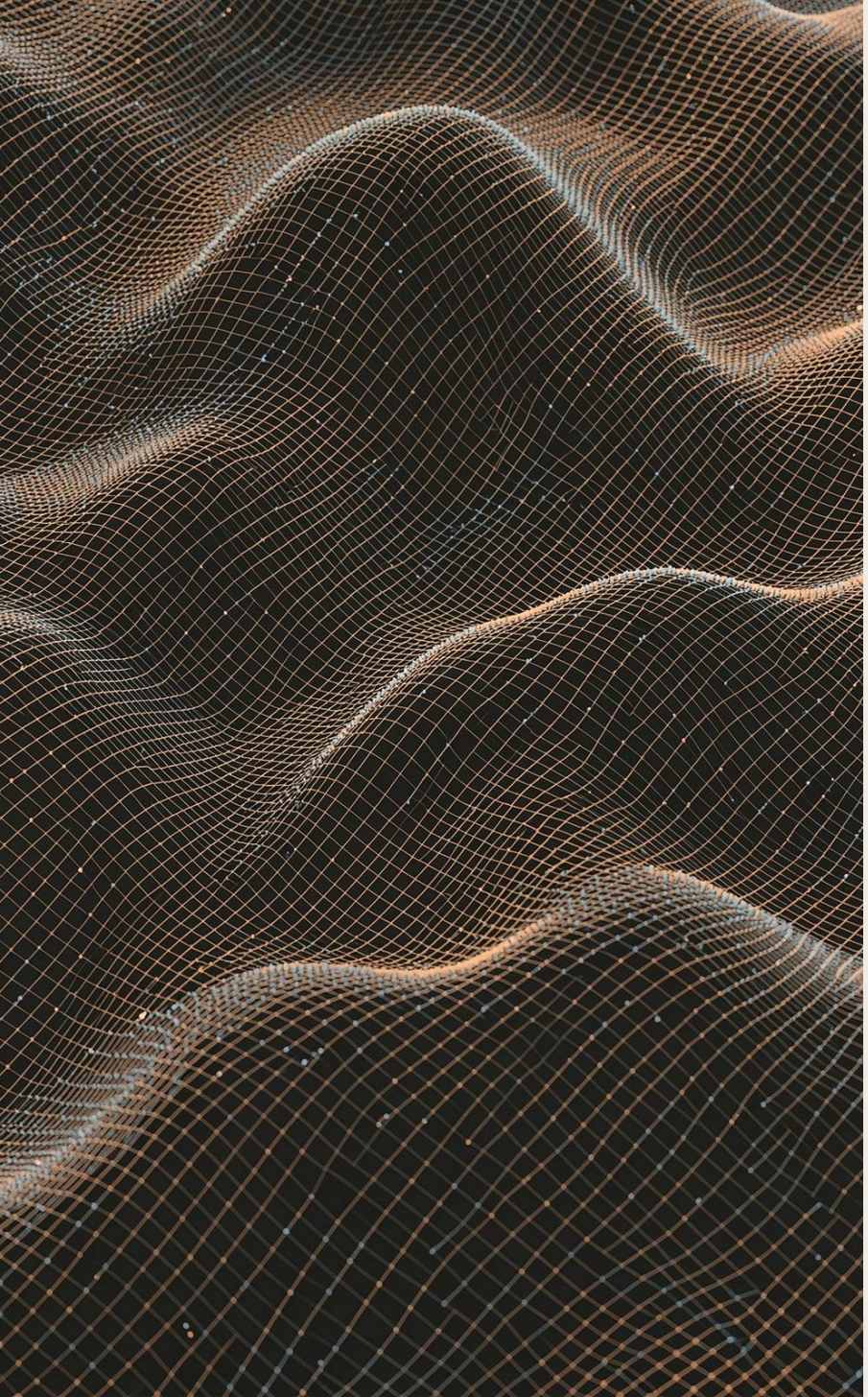
Notational

Shared visual standards like UML class diagrams

Procedural

Standard template documents and formal code review checklists

Made with GAMMA



Decision Records Preserve Critical System Design Rationale

1 **Rationale**

The justification and context behind our architectural decisions

2 **Captured Context**

Problems, alternatives evaluated, evaluation criteria, debates

3 **Beyond the Model**

Code and UML diagrams represent the final solution model, not the rationale

4 **The *Why* Matters**

Understanding the *why* is essential to modify legacy systems safely

Made with GAMMA

SCM Baselines Manage Configuration Control Paths

Software Configuration Management (SCM) is the process of monitoring and controlling changes in work products. A disciplined SCM practice is the backbone of any stable engineering workflow.

Configuration Item
Any versioned file or document

Baseline
Formally reviewed and frozen state



Version
Evolutionary state enabling rollbacks

Controlled Release
Authorized distribution of baseline

1

SCM

The process of monitoring and controlling changes in work products

2

Configuration Item

Any independently versioned file, document, or model

3

Version

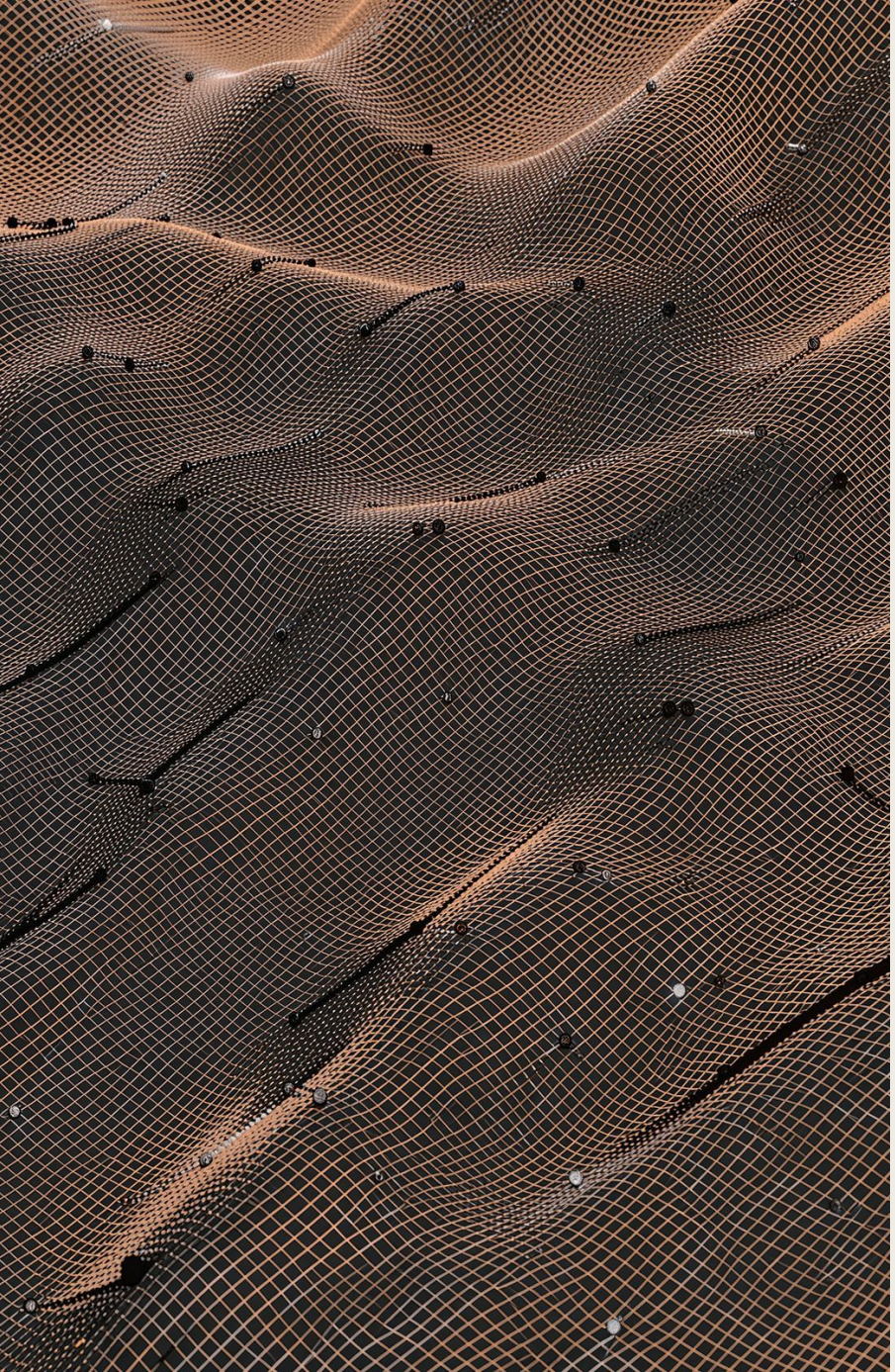
An evolutionary state of an item; supports rollbacks

4

Baseline

A formally reviewed, approved, and frozen system state

Made with GAMMA



ASSESSMENT

Selecting Process Models under Safety Constraints

Scenario: You are designing safety-critical flight control software. **Constraints:** Fixed budget, rigid safety verification, high complexity.

Question: Which development model is most appropriate?

A. Pure Sequential Waterfall

Linear phases with no iteration; insufficient for high-risk complexity

B. Risk-Based Prescriptive Model ✓

Structured risk management with formal verification gates — the correct choice

C. Decentralized Issue-Based Model

Lacks the formal control required for safety-critical certification

Made with GAMMA

ASSESSMENT

Mapping TicketDistributor System Requirements

Classify these three TicketDistributor statements as **Functional** or **Nonfunctional**:



Statement 1

"Enable a traveler to buy weekly passes."

→ **Functional** — describes a specific system capability or behavior



Statement 2

"The TicketDistributor must be written in Java."

→ **Nonfunctional** — an implementation constraint, not a behavior



Statement 3

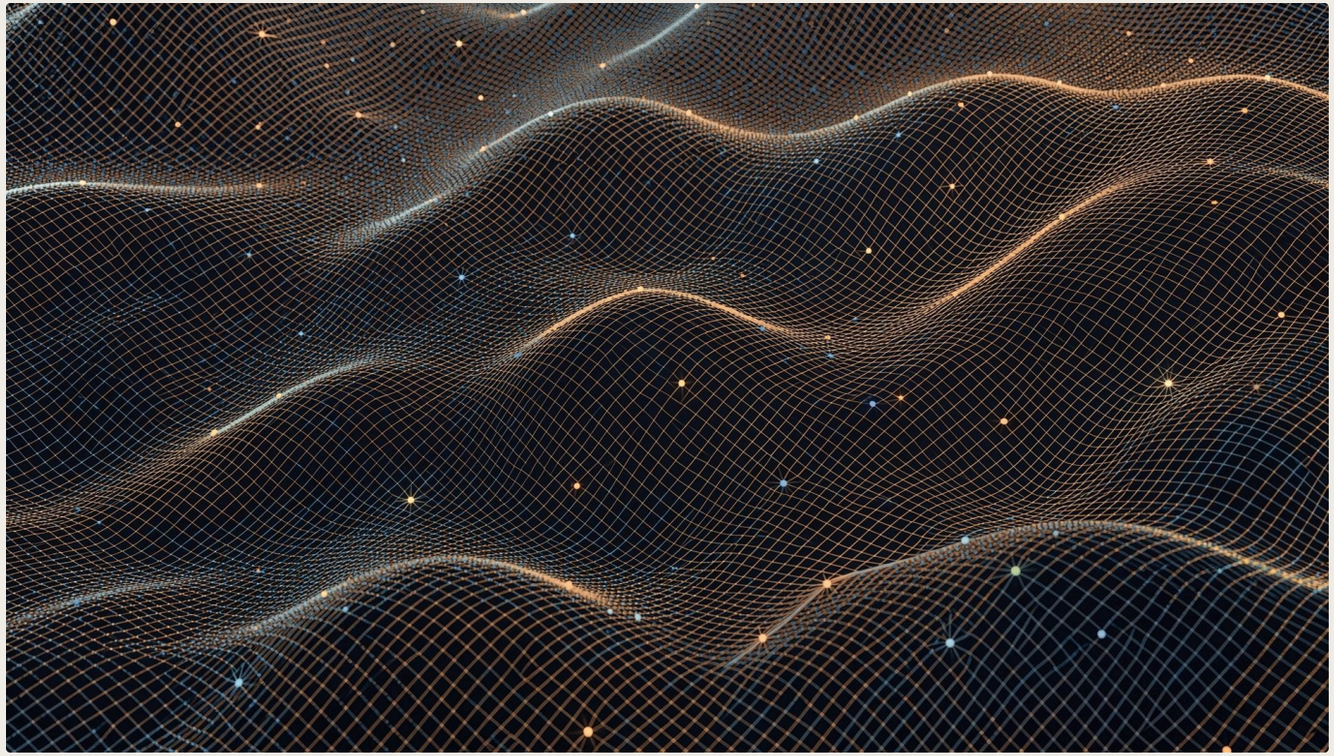
"The TicketDistributor must always be available."

→ **Nonfunctional** — a quality-of-service / reliability requirement

Made with GAMMA

ASSESSMENT

Restructuring Flat Capstone Mesh Teams



Scenario

A 15-person capstone team operates as a flat, decentralized peer mesh.

The Issue

The team suffers from **high communication overhead** and **decision paralysis**.

Task

Identify the primary risk and propose the first convention to introduce.

⚠ Unstructured communication in a 15-person mesh produces up to **105 unique communication channels** — a critical coordination failure point.

Made with GAMMA

Abstract Models Systematically Defeat Project Volatility

1

Acknowledge Complexity

Software is uniquely complex and subject to constant, rapid change

2

Apply Discipline

Raw programming is insufficient; we must apply disciplined engineering processes

3

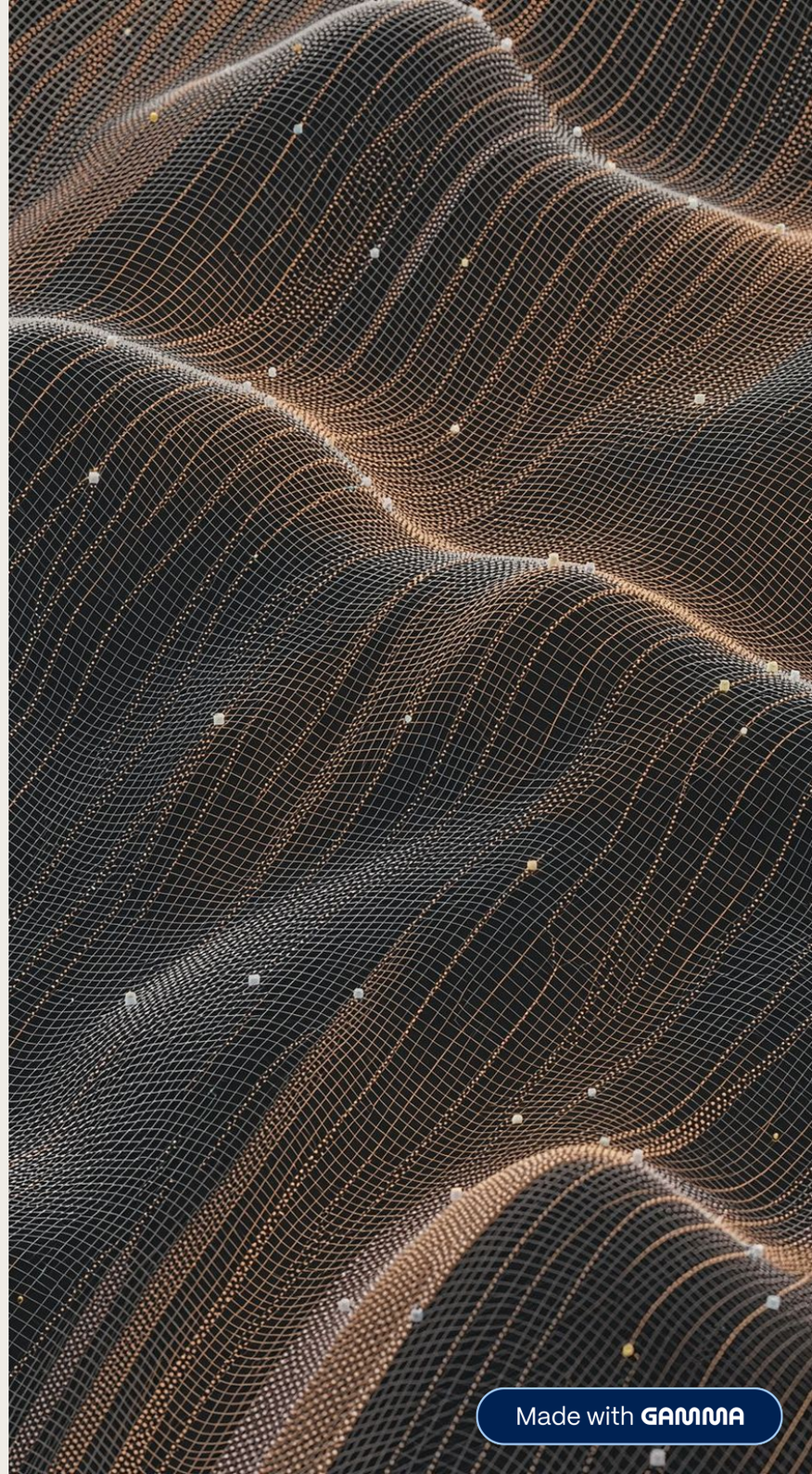
Model with UML

UML modeling is our primary weapon to ignore noise and focus on key design tasks

4

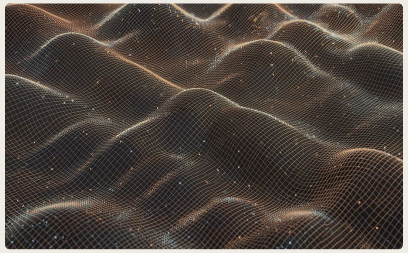
Enforce Conventions

Shared team conventions eliminate costly communication bottlenecks



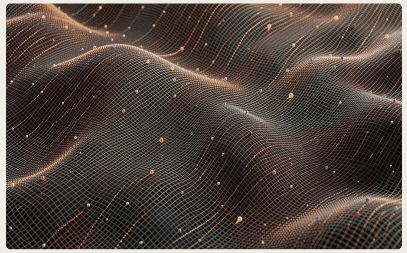
Made with GAMMA

Three Core Pillars Anchor Sustainable System Architectures



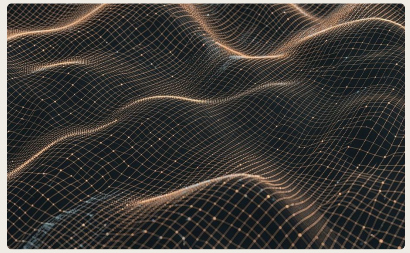
Modeling Pillar

Abstracting systems to resolve complex design questions (UML)



Process Pillar

Structuring activities to manage risk and change (Waterfall vs. Agile)



People Pillar

Defining clear roles and team conventions to manage communication

Made with GAMMA

HOMEWORK

Analyzing the Software Paradox Delay

The Paradox

Aircraft and bridges have millions of parts but are delivered on time.

The Case

Word for Windows (1989) was only **249,000 LOC**, but was **4 years late**.

Your Task

Write a **1-page paper** analyzing why software is uniquely prone to delay.

Ground your arguments in the concepts of **complexity** and **change**.

> Consider: What makes software fundamentally different from physical engineering artifacts? Why do traditional project management techniques fail to transfer?

Made with GAMMA

Bibliographic References and Future Design Readings

Primary Reading

Bruegge & Dutoit, *Object-Oriented Software Engineering*, 3rd Ed., Chapter 1

Historical Reference

Brooks, F. *The Mythical Man-Month*, 1995

Case Reference

Neumann, P. *Computer-Related Risks*, 1995

Epistemology Reference

Popper, K. *Objective Knowledge*, 1992

Made with GAMMA